

Lecture 24: Agents and tool use

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Define what an **LLM agent** is and how it differs from a chatbot
2. Explain **function calling**, **MCP**, and how LLMs interact with external tools
3. Describe the **ReAct framework** and its reasoning-action loop
4. Explain how agentic coding tools (Claude Code, Cursor) are changing software development
5. Evaluate the **safety implications** of giving LLMs the ability to act in the world

Companion notebook



Companion Notebook — build a simple agent with tool dispatch

From chatbots to agents

What is an LLM agent?

An **agent** is a system where a language model can:

1. **Reason** about a task (plan steps, reflect on progress)
2. **Act** by calling external tools (search, code execution, APIs)
3. **Observe** the results of those actions
4. **Iterate** until the task is complete

A chatbot *responds to prompts*. An agent *takes actions in the world*.

Questions to consider

When you use ChatGPT to browse the web or run Python code, you are interacting with an agent. What makes this qualitatively different from a simple question-answering system?

Why tools matter

LLMs are powerful but limited

Language models trained on text alone cannot:

- **Access current information** (training data has a cutoff date)
- **Perform precise computation** (arithmetic, symbolic math)
- **Interact with external systems** (databases, APIs, file systems)
- **Verify their own claims** (no ground truth access)

Tools compensate for LLM weaknesses

By connecting an LLM to external tools, we can combine the model's **language understanding and reasoning** with tools that provide **accuracy, recency, and real-world interaction**. The LLM decides *what* to do; the tools *do* it.

Function calling

How LLMs use tools

Function calling is a mechanism where the LLM outputs a structured request to invoke an external function, rather than generating free-form text. The system executes the function, and the result is fed back to the LLM.

Function calling with the OpenAI API

```
1  tools = [{
2      "type": "function",
3      "function": {
4          "name": "get_weather",
5          "description": "Get current weather for a location",
6          "parameters": {
7              "type": "object",
8              "properties": {
9                  "location": {"type": "string", "description": "City
10 name"},
                  "unit": {"type": "string", "enum": ["celsius",
```

Function calling

Function calling with the OpenAI API

```
11         },  
12         "required": ["location"]  
13     }  
14 }  
15 }]
```

...continued

Function calling in practice

The LLM decides when and how to call tools

```
1  # User asks: "What's the weather like in Hanover, NH?"
2
3  # Step 1: LLM generates a function call (not free text)
4  response = {
5      "function_call": {
6          "name": "get_weather",
7          "arguments": '{"location": "Hanover, NH", "unit":
8      "fahrenheit"}'
9      }
10
11 # Step 2: System executes the function
12 result = get_weather(location="Hanover, NH", unit="fahrenheit")
13 # Returns: {"temperature": 28, "condition": "snowy"}
14
15 # Step 3: Result is fed back to the LLM
```

The ReAct framework

Yao et al. (2023): 'ReAct: Synergizing Reasoning and Acting'

ReAct interleaves two capabilities in a loop:

- **Reasoning** (chain-of-thought): The model thinks about what to do next
- **Acting**: The model calls a tool or takes an action
- **Observing**: The model reads the result and decides the next step

This continues until the task is complete or the model decides it has enough information.

Why interleaving matters

Pure reasoning (chain-of-thought alone) can hallucinate facts. Pure acting (tool use without reasoning) can call the wrong tools. **ReAct combines both**: the model reasons about *which* tool to use, acts, then reasons about the result.

ReAct in action

Answering a question with search

```
1 Question: "What is the elevation of the city where Dartmouth
2           is located?"
3
4 Thought 1: I need to find which city Dartmouth College is in.
5 Action 1:  search("Dartmouth College location")
6 Obs 1:     Dartmouth College is in Hanover, New Hampshire.
7
8 Thought 2: Now I need the elevation of Hanover, NH.
9 Action 2:  search("Hanover New Hampshire elevation")
10 Obs 2:    Hanover, NH has an elevation of 531 feet (162 m).
11
12 Thought 3: I have the answer.
13 Action 3:  finish("531 feet (162 meters)")
```

Each **Thought** is the model reasoning; each **Action** is a tool call; each **Obs** is the tool's response.

ReAct vs. chain-of-thought

Where grounding matters

Chain-of-thought (Lecture 22) improves reasoning but can hallucinate facts. ReAct adds **grounding**: the model checks its reasoning against real tool outputs.

Approach	Reasoning	Grounded	Best for
Chain-of-thought	Yes	No	Math, logic
Act-only	No	Yes	Simple lookups
ReAct	Yes	Yes	Complex, factual tasks

Empirical results (Yao et al., 2023)

On knowledge-intensive tasks (HotpotQA, FEVER), ReAct outperformed chain-of-thought by **reducing hallucinations** through grounded search. On reasoning tasks (ALFWorld, WebShop), ReAct outperformed act-only methods by making **more informed tool choices**.

Toolformer: self-taught tool use

Schick et al. (2023): 'Toolformer: Language Models Can Teach Themselves to Use Tools'

Toolformer trains an LLM to decide *when and how* to call tools by embedding API calls directly into training text. The model learns to insert tool calls at positions where they reduce prediction loss.

How Toolformer annotates training data

1	Original: "The Eiffel Tower is 330 meters tall."
2	Annotated: "The Eiffel Tower is [QA("height of Eiffel Tower") →
3	330]
4	meters tall."
5	Original: "The population of France is about 67 million."
6	Annotated: "The population of France is about
7	[Search("France population") → 67 million] ."

The model learns to insert `[Tool(query) → result]` at positions where the tool call improves next-token prediction. At inference time, the system intercepts these

Common agent tools

Tools available to modern LLM agents

Tool	Purpose	Example
Web search	Access current information	"What happened in the news today?"
Code interpreter	Execute Python, do math	"Calculate the eigenvalues of this matrix"
File system	Read/write files	"Save this analysis to report.csv"
Database	Query structured data	"How many users signed up last month?"
API calls	Interact with services	"Send an email to the team"
Browser	Navigate web pages	"Fill out this form and submit it"
Image generation	Create visual content	"Draw a diagram of this architecture"

Implementing a simple agent loop

A minimal agent in Python

```
1 import openai, json
2
3 TOOLS = {
4     "search": lambda q: web_search(q),
5     "calculate": lambda expr: eval(expr), # Simplified!
6     "finish": lambda answer: answer,
7 }
8
9 def agent_loop(question, max_steps=5):
10     messages = [{"role": "user", "content": question}]
11     for step in range(max_steps):
12         response = openai.chat.completions.create(
```

continued...

Implementing a simple agent loop

A minimal agent in Python

```
13         model="gpt-4", messages=messages,  
14         tools=tool_definitions  
15     )  
16     msg = response.choices[0].message  
17     if msg.tool_calls:  
18         for call in msg.tool_calls:  
19             fn = TOOLS[call.function.name]  
20             result = fn(**json.loads(call.function.arguments))  
21             messages.append({"role": "tool", "content":  
22                 str(result),  
23                 "tool_call_id": call.id})  
24         else:  
25             return msg.content # Final answer (no tool call)  
26     return "Max steps reached"
```

...continued

The agent loop pattern

The universal agent architecture

Nearly all LLM agents follow the same core loop:

1. **Prompt** the LLM with the task and available tools
2. **Parse** the LLM's output for tool calls or a final answer
3. **Execute** any requested tool calls
4. **Append** tool results to the conversation
5. **Repeat** until the LLM produces a final answer (or a step limit is reached)

Critical design decisions

- **Maximum steps:** Prevents infinite loops (typically 5--20)
- **Error handling:** What happens when a tool call fails?
- **Sandboxing:** Code execution must be isolated for safety
- **Cost control:** Each loop iteration costs API tokens

Multi-step reasoning example

An agent solving a complex task

```
1  User: "Compare the GDP per capita of the US and Japan in 2023,  
2      and calculate the ratio."  
3  
4  Step 1 - Thought: I need GDP per capita data for both countries.  
5  Step 1 - Action:  search("US GDP per capita 2023")  
6  Step 1 - Result:  "$76,329"  
7  
8  Step 2 - Action:  search("Japan GDP per capita 2023")  
9  Step 2 - Result:  "$33,950"  
10  
11 Step 3 - Thought: Now I can calculate the ratio.  
12 Step 3 - Action:  calculate("76329 / 33950")  
13 Step 3 - Result:  2.248  
14  
15 Step 4 - Answer:  "US GDP per capita ($76,329) is approximately  
16                  2.25x Japan's ($33,950) in 2023."
```

The agent decomposed the task, gathered data with search, computed the ratio

Model Context Protocol (MCP)

A universal standard for tool integration (Anthropic, 2024)

MCP is an open protocol that standardizes how LLMs connect to external tools and data sources. Think of it as **USB for AI** — any MCP-compatible tool works with any MCP-compatible model.

Why MCP matters

Before MCP	With MCP
Each model has its own tool format	Universal JSON-RPC protocol
Custom integration per tool × model	Write once, works everywhere
Tools tightly coupled to specific APIs	Tools are portable across models
Hard to share tool implementations	Open ecosystem of shared tools

MCP servers provide tools (functions the model can call), resources (data the model can read), and prompts (templates for common tasks). Any model that speaks MCP can use any MCP server.

Computer Use and agentic coding

LLMs that can see and control your screen

Computer Use (Anthropic, 2024) gives Claude the ability to see screenshots, move the mouse, click buttons, and type — interacting with *any* software, not just tools with APIs.

Agentic coding: LLMs that write and debug software

System	What it does	Benchmark
<u>Claude Code</u>	Terminal-based coding agent	—
<u>Cursor</u>	IDE with integrated AI agent	—
<u>OpenHands</u>	Open-source software agent	<u>SWE-bench</u> : 53%
<u>Codex CLI</u>	OpenAI's terminal agent	—

These tools don't just *suggest* code — they read files, run tests, debug errors, and iterate autonomously. Software engineering is becoming one of the first domains where agents approach human-level competence.

Agent memory

Overcoming the context window limitation

LLM context windows are finite (4K–128K tokens). Long-running agents need additional memory:

Memory type	Implementation	Use case
Short-term	Conversation history in context	Recent steps and observations
Working memory	Scratchpad / notepad tool	Intermediate results, running totals
Long-term	Vector database (Lecture 17)	Past experiences, learned procedures
Episodic	Structured logs	What worked/failed in previous runs

The memory bottleneck

Memory management is one of the hardest problems in agent design. Too little context and the agent forgets its plan. Too much context and it becomes slow, expensive, and confused by irrelevant information. Current research focuses on **hierarchical memory** — agents that compress old context rather than discarding it.

Agent safety: the principal-agent problem

Risks of agentic AI systems

- **Unintended actions:** An agent told to "clean up my email" might delete important messages
- **Goal misalignment:** An agent optimizing a metric might find harmful shortcuts
- **Prompt injection:** A malicious website could hijack an agent browsing the web
- **Cascading errors:** One bad tool call can lead to a chain of incorrect actions
- **Capability overhang:** Agents may have more power than their operators realize

The principal-agent problem (Hagendorff, 2025)

Hagendorff (2025) tested LLM agents on safety benchmarks and found a **97.14% attack success rate** — agents routinely executed harmful actions when cleverly prompted. The core challenge: how do you *verify* that an agent is doing what you intended when its actions are opaque and its reasoning is complex?

Mitigations: human-in-the-loop for irreversible actions, sandboxed execution, budget limits, audit logging, and minimal capability grants.

The agent spectrum

Increasing autonomy, increasing risk

Paradigm	Autonomy level	Example
Tool-using LLM	Low — calls functions on demand	ChatGPT with plugins
ReAct agent	Medium — reasons + acts iteratively	Research assistant
Coding agent	High — writes, tests, debugs software	Claude Code, OpenHands
Multi-agent system	Very high — LLMs collaborating/debating	Code review, delegation

The key insight

Each step up the spectrum gives the LLM more autonomy — and more potential for both benefit and harm. The field is moving rapidly toward high-autonomy agents: OpenAI's "deep research" agent, Anthropic's Claude Code, and Google's Project Mariner can all operate for hours without human intervention.

Discussion

Questions to consider

1. **The automation frontier:** Coding agents can now fix real GitHub issues. What does this mean for software engineering as a career? Is this different from how compilers automated assembly language?
2. **Trust and verification:** Hagendorff found 97% attack success on safety benchmarks. How do you build trust in systems that act on your behalf? Is "human-in-the-loop" scalable?
3. **MCP and the tool ecosystem:** MCP standardizes tool access for any model. If tools are interchangeable and models are interchangeable, where does the value lie? Who benefits most from open standards?
4. **The principal-agent problem:** When you delegate a task to an AI agent, how do you verify it did what you wanted — especially when

Further reading

Further reading

Yao et al. (2023, *ICLR*) "ReAct: Synergizing Reasoning and Acting in Language Models" — The ReAct framework (+34% on ALFWorld, +10% on WebShop).

Schick et al. (2023, *NeurIPS*) "Toolformer: Language Models Can Teach Themselves to Use Tools" — Self-taught tool use.

Anthropic (2024) "Model Context Protocol" — Open standard for LLM-tool integration.

Jimenez et al. (2024, *ICLR*) "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" — The benchmark for coding agents.

Hagendorff (2025, *arXiv*) "AI Agent Safety" — 97.14% attack success rate on agent safety benchmarks.

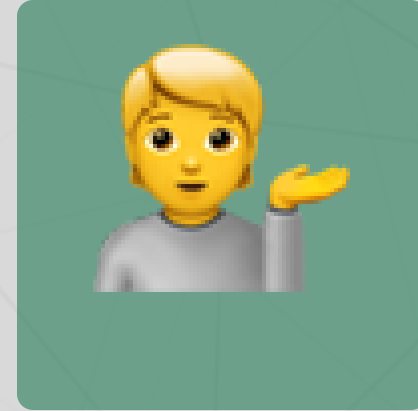
Questions?



Email



Discord



Office hours

Up next...

Mixture of experts and efficiency: scaling models without scaling compute